Clearance — POC Specification

Clearance — POC Specification

What It Is

Clearance is a **headless MCP server** that gives any MCP-capable host — Claude Desktop, VS Code Copilot, Cursor, or anything else — the ability to review code conversationally.

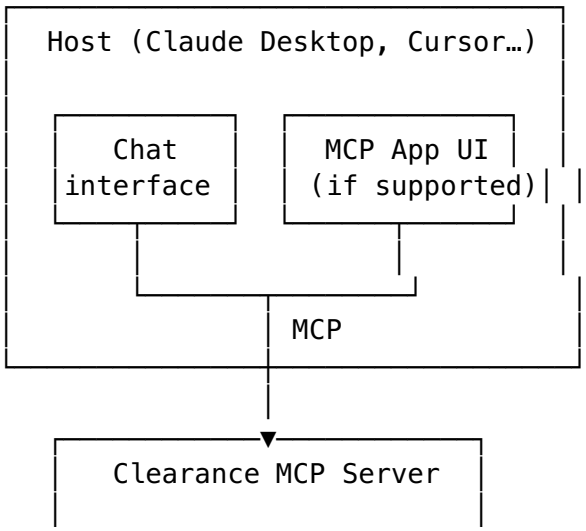
It has no UI of its own. The host provides the chat interface. Clearance provides the tools: diff access, structured analysis, comment staging, and GitHub submission. The agent in the host orchestrates them.

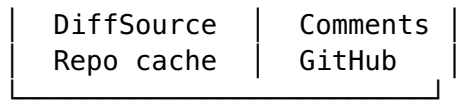
This is the key architectural decision: Clearance does not compete with or duplicate the host. It plugs in to whatever tool the developer is already working in.

Where the host supports the MCP Apps spec (SEP-1865), Clearance surfaces a rich diff viewer as an embedded iframe alongside the conversation. Where it doesn't, everything works via text — line references, structured tool responses, and plain conversation. The experience degrades gracefully; the workflow never breaks.

It works identically with GitHub PRs and local git diffs.

The Headless Model





The host model drives the review. Clearance supplies the data and actions. This means Clearance works today in every MCP-capable host, and gets better automatically as hosts improve their MCP App support.

V1 Scope

Feature	Status
GitHub PR checkout via OAuth + bare clone + worktree	v1
Local diff support (`git diff <base>..<head>`)	v1
Diff parsing and structured hunk model	v1
Opening briefing from the agent	v1
Conversational Q&A with line-level diff access	v1
MCP App diff viewer (React + Vite, where host supports it)	v1
Staged comments panel + GitHub submission	v1
Persistent review instructions (global + per-repo)	v1
Review chapters (large PR decomposition)	post-MVP
Surprise detector (anomaly flagging)	post-MVP
“Brief me in 2 minutes” mode	post-MVP
Learns your patterns over time	post-MVP

Installation

Distributed via npm, invoked via `npx`. Zero install — users add one entry to their MCP config:

```
{
  "mcpServers": {
    "clearance": {
      "command": "npx",
      "args": ["clearance"]
    }
  }
}
```

One-time GitHub auth:

```
npx clearance auth
```

That’s it. The host picks up the tools automatically on next restart.

Authentication

GitHub auth uses the **OAuth device flow** — `clearance auth` opens a browser prompt, the user approves, and the token is stored in the **system keychain** via `keytar`. Never written to disk or env vars. Subsequent server starts pull from the keychain silently.

Diff Source Abstraction

The core pipeline is built on a `DiffSource` interface. All analysis, tool responses, and UI rendering operate against this interface — nothing knows or cares whether the source is a GitHub PR or a local git diff.

```
interface DiffSource {
  getMetadata(): Promise<DiffMetadata>        // title,
        description, author, linked issue
  getDiff(): Promise<ParsedDiff>              // structured hunk
        model
  getFileContent(path: string, ref: string): Promise<string>
}

class GitHubPRDiffSource implements DiffSource { ... }
class LocalGitDiffSource implements DiffSource { ... }
```

`LocalGitDiffSource` accepts any two git refs — `main..HEAD`, `HEAD~1..HEAD`, `v1.2.0..v1.3.0`, any commit SHA. This makes local review as powerful as PR review: you can review your own branch before opening a PR, audit a release diff, or inspect any two points in history.

Repo Management

- One **bare clone** per repo at `~/.clearance/repos/<owner>/<repo>.git`
- Each PR gets a **git worktree** (`git worktree add`) off the fetched PR ref — isolated, cheap, disposable
- Worktrees created on session start, removed on session end
- Bare clones are shallow (`--depth=1`), refreshed via `git fetch` on demand
- Local diff reviews use the current working directory directly — no clone needed

---

Diff Parsing

Raw git diff is parsed into a typed model via **`parse-diff`**:

```
interface ParsedDiff {
  files: ParsedFile[]
}
```

```ts
interface ParsedFile {
  path: string
  hunks: Hunk[]
}

interface Hunk {
  oldStart: number
  newStart: number
  lines: DiffLine[]
}

interface DiffLine {
  type: 'add' | 'del' | 'context'
  oldNumber?: number
  newNumber?: number
  content: string
}
```

Every line number reference in conversation — whether from the agent or the user — maps back to this structure. The agent never string-munges raw patch format.

MCP Tool Surface

Discrete, composable tools. The agent orchestrates them to produce the conversational review experience:

Tool	Description
`start_review`	Initialise session, checkout diff source, return metadata + opening briefing
`get_diff_summary`	Structured summary of all changed files and hunk counts
`get_file_hunk`	Return lines N–M of a specific file from the diff
`get_file_content`	Full file content at a given ref
`search_diff`	Find diff lines matching a pattern
`stage_comment`	Queue a comment at a file + line
`list_staged_comments`	Return all pending comments
`edit_staged_comment`	Update a queued comment
`discard_staged_comment`	Remove a queued comment
`submit_review`	Post all staged comments to GitHub as a review

MCP App UI

Where the host supports MCP Apps (SEP-1865), Clearance renders a diff viewer as a sandboxed iframe (`text/html;profile=mcp-app`), communicating bidirectionally with the server via `postMessage` JSON-RPC.

Built with **React + Vite** to keep the build simple and the ecosystem familiar.

Panels

Diff viewer — `react-diff-viewer-continued`. Unified by default, toggle to split. Displays only the hunks the agent has surfaced — never the whole diff unless asked. Line numbers are preserved from the parsed model. Clicking a line inserts a reference (e.g. `index.ts:42`) into the chat input.

Staged comments — lists all pending comments with file, line, and body. Editable and discardable inline. Submit Review posts to GitHub.

Session header — PR title or ref range, author, branch, linked issue.

Graceful degradation

Hosts without MCP App support get the full review experience via text: the agent references files and lines explicitly, tool responses return formatted text, and comments are staged and submitted the same way. Nothing breaks — the diff viewer is an enhancement, not a dependency.

Review Instructions

Plain markdown files injected into the agent's context at session start:

Path	Scope
`~/.clearance/instructions.md`	Global — all reviews, all repos
`.clearance/instructions.md`	Per-repo — committed to the repo root, shared with the team

`clearance config` opens the right file in `$EDITOR`. The files are the source of truth — no magic config store.

Opening Briefing

On `start_review` the agent receives the full diff summary, PR metadata or ref range, linked issue body (if any), and review instructions. It opens with:

1. What this change is doing in one paragraph
2. The 2–3 things the reviewer is most likely to care about

3. Anything that surprised it — pattern breaks, unexpected file touches, description vs. diff size mismatch

---

Stack

Concern	Choice
Language	TypeScript
Distribution	npx
MCP SDK	`@modelcontextprotocol/sdk`
GitHub API	Octokit (`@octokit/rest`)
Auth storage	`keytar` (system keychain)
Diff parsing	`parse-diff`
Repo/worktree management	`simple-git`
MCP App UI	React + Vite
Diff viewer component	`react-diff-viewer-continued`
MCP App rendering	MCP Apps spec (SEP-1865)

Open Questions

- Should `.clearance/instructions.md` be gitignored by default (personal) or committed (team)? Leaning committed — team benefits from shared review context.
- Should `submit_review` support “Request Changes” and “Approve” states in v1, or just “Comment”?
- Rate limiting strategy for GitHub API on large PRs.
- How to handle MCP hosts that don’t support MCP Apps capability negotiation gracefully — auto-detect and skip, or warn the user once?